BIOST 2155: Assignment 3 — Solutions

Answer key

Released: 2026-10-16. **Due by**: 2026-10-30, 11:59 PM

Instructions

```
knitr::opts_chunk$set(echo = TRUE, eval = TRUE,
                      fig.height = 4, fig.width = 7, fig.align = "center",
                      message = FALSE, warning = FALSE)
```

This assignment covers **Classification II** (support vector machines) and **Clustering I** (k-means, cluster evaluation). It uses two data sets: `Wisconsin Breast Cancer` (`assign3_data.csv`) for Part A, and `Palmer Penguins` (`penguins.csv`) for Part B. Part C is a simulation study.

Theory (12 points)

T1. The k-means objective and Lloyd's algorithm (7 + 5 = 12 points)

This problem involves the K-means clustering algorithm. Let $C_1, \ldots, C_K$ be a clustering of n data points. The objective function of K-means is given by

$$\min_{C_1,\ldots,C_K} \left\{ \sum_{k=1}^{K} n(C_k)^{-1} \sum_{j,j' \in C_k} \sum_{i=1}^{p} (x_{ji} - x_{j'i})^2 \right\},$$

where $n(C_k)$ is the number of data points in C_k.

(a) (7 points) Given that the center of C_k is $\boldsymbol{\mu}_k = n(C_k)^{-1} \sum_{j \in C_k} \boldsymbol{x}_j$, prove that the objective function above is equivalent to

$$\min_{(C_1,\ldots,C_K)} \sum_{k=1}^{K} \left\{ 2 \cdot \sum_{j \in C_k} \sum_{i=1}^{p} (x_{ji} - \mu_{ki})^2 \right\}.$$

(b) (5 points) On the basis of the identity proved in (a), argue that Lloyd's clustering algorithm decreases the above objective function at each iteration.

Show your working — you may upload handwritten answers for this problem.

(a) It is enough to prove the identity for one cluster and one coordinate, then sum over $i = 1, \ldots, p$ and $k = 1, \ldots, K$. Fix a cluster C with $m = n(C)$ points and a coordinate i, and write $\mu_i = m^{-1} \sum_{j \in C} x_{ji}$. Add

and subtract μ_i inside the square:

$$\begin{aligned}\sum_{j,j'\in C}(x_{ji}-x_{j'i})^2 &= \sum_{j,j'\in C}\big[(x_{ji}-\mu_i)-(x_{j'i}-\mu_i)\big]^2\\ &= \sum_{j,j'}(x_{ji}-\mu_i)^2+\sum_{j,j'}(x_{j'i}-\mu_i)^2-2\sum_{j,j'}(x_{ji}-\mu_i)(x_{j'i}-\mu_i)\\ &= m\sum_{j\in C}(x_{ji}-\mu_i)^2+m\sum_{j'\in C}(x_{j'i}-\mu_i)^2-2\Big[\sum_{j\in C}(x_{ji}-\mu_i)\Big]\Big[\sum_{j'\in C}(x_{j'i}-\mu_i)\Big]\\ &= 2m\sum_{j\in C}(x_{ji}-\mu_i)^2,\end{aligned}$$

because $\sum_{j\in C}(x_{ji}-\mu_i)=m\mu_i-m\mu_i=0$, so the cross term vanishes. Dividing by $m=n(C)$:

$$n(C)^{-1}\sum_{j,j'\in C}(x_{ji}-x_{j'i})^2=2\sum_{j\in C}(x_{ji}-\mu_i)^2.$$

Summing over i and over the K clusters, the two objective functions are equal for **every** clustering $C_1,\dots,C_K$. So they have the same minimizer, and the two problems are equivalent. ■

Note: the sum over j,j' is over ordered pairs and includes $j=j'$ (those terms are zero). Summing over unordered pairs with the matching factor is equally valid.

(b) By (a), the objective equals $2\,F(C,\mu(C))$, where for any clustering C and any set of centres $m_1,\dots,m_K$ we define

$$F(C,m)=\sum_{k=1}^{K}\sum_{j\in C_k}\|x_j-m_k\|^2,$$

and $\mu(C)$ are the cluster means. Lloyd's algorithm alternates two steps, and neither can increase F:

1. **Update step (clusters fixed, move the centres).** For one cluster and any point m, $\sum_{j\in C_k}\|x_j-m\|^2=\sum_{j\in C_k}\|x_j-\mu_k\|^2+n(C_k)\,\|\mu_k-m\|^2$ (same add-and-subtract argument; the cross term is again zero). This is smallest at $m=\mu_k$. So replacing each centre with its cluster mean cannot increase F.
2. **Assignment step (centres fixed, move the points).** Each point moves to its nearest centre, so each term $\|x_j-m_{c(j)}\|^2$ goes down or stays the same. So F cannot increase.

After a full iteration we are back at a pair $(C^{\text{new}},\mu(C^{\text{new}}))$, and by (a) the objective is $2F(C^{\text{new}},\mu(C^{\text{new}}))\le 2F(C^{\text{old}},\mu(C^{\text{old}}))$. The objective therefore never increases. It strictly decreases in any iteration where some point changes cluster to a strictly closer centre. Otherwise no point moves and the algorithm has converged. There are finitely many partitions, so the algorithm stops after finitely many steps. It stops at a **local** minimum, which need not be the global one. That is why we use many random starts. ■

Application, Part A: Classification II — Support Vector Machines (35 points)

Dataset: `Wisconsin Breast Cancer`, saved as `assign3_data.csv`.

The dataset contains `n = 569` observations on breast tumors, with 10 features computed from a digitized image of a fine needle aspirate (FNA) of a breast mass. These describe characteristics of the cell nuclei present in the image. The target label `diagnosis` (`M`: malignant / `B`: benign) indicates whether the tumor is malignant or benign. Your goal is to build a support vector classifier to distinguish malignant from benign tumors. **Please set a random seed (`set.seed(42)`) before any step involving data splitting or randomization.**

```
rm(list = ls())
pacman::p_load(tidyverse, caret, e1071, pROC, kernlab)
set.seed(42)
```

```
data <- read_csv("assign3_data.csv") |> dplyr::select(-1) |> drop_na()
data$diagnosis <- factor(data$diagnosis, levels = c("B", "M"))
```

A1. Data Preparation (3 + 3 = 6 points)

1. Split the data into a training set (75%) and a test set (25%), stratified by `diagnosis`.

```
set.seed(42)
idx   <- createDataPartition(data$diagnosis, p = 0.75, list = FALSE)
train <- data[idx, ]
test  <- data[-idx, ]
rbind(train = table(train$diagnosis), test = table(test$diagnosis))
```

```
##         B   M
## train 268 159
## test   89  53
```

427 training and 142 test tumors. Stratifying keeps the malignant share (about 37%) the same in both sets.

2. Support vector classifiers are sensitive to feature scale. Center and scale the 10 predictors in both the training and test sets (fit the scaling on the training set only, then apply it to the test set).

```
pp      <- preProcess(train, method = c("center", "scale"))  # learns train means/SDs
train_s <- predict(pp, train)
test_s  <- predict(pp, test)   # the SAME train means/SDs applied to the test set
round(colMeans(train_s[, -1]), 3)   # exactly 0 on train ...
```

```
##            radius_mean           texture_mean         perimeter_mean
##                      0                      0                      0
##              area_mean        smoothness_mean       compactness_mean
##                      0                      0                      0
##         concavity_mean    concave_points_mean          symmetry_mean
##                      0                      0                      0
## fractal_dimension_mean
##                      0
```

```
round(colMeans(test_s[, -1]), 3)    # ... close to, but not exactly, 0 on test
```

```
##            radius_mean           texture_mean         perimeter_mean
##                 -0.043                  0.021                 -0.042
##              area_mean        smoothness_mean       compactness_mean
##                 -0.050                 -0.083                  0.023
##         concavity_mean    concave_points_mean          symmetry_mean
##                 -0.025                 -0.050                  0.011
## fractal_dimension_mean
##                  0.072
```

The test means are close to 0 but not exactly 0. That is the expected result of using the training means: the test set never informs the preprocessing.

Note: scale using the training-set means and SDs, then apply them to the test set. Scaling the full data before splitting, or scaling train and test separately, either leaks information or puts the two sets on different scales.

A2. Linear Support Vector Classifier (6 + 4 = 10 points)

1. Using `e1071::tune()` (or `caret::train` with `method = "svmLinear"`), perform 10-fold cross-validation to select the optimal cost parameter `C` over a grid of your choice (e.g. `c(0.01, 0.1, 1, 5, 10, 100)`). Report the chosen `C` and the number of support vectors in the final model.

```
ctrl <- trainControl(method = "cv", number = 10)
set.seed(42)
svm_lin <- train(diagnosis ~ ., data = train_s, method = "svmLinear",
                 trControl = ctrl,
                 tuneGrid = expand.grid(C = c(0.01, 0.1, 1, 5, 10, 100)))
svm_lin$results[, c("C", "Accuracy")]
```

```
##       C  Accuracy
## 1 1e-02 0.9272425
## 2 1e-01 0.9343300
## 3 1e+00 0.9366002
## 4 5e+00 0.9318937
## 5 1e+01 0.9295681
## 6 1e+02 0.9366002
```

```
c(chosen_C = svm_lin$bestTune$C, n_support_vectors = svm_lin$finalModel@nSV)
```

```
##          chosen_C n_support_vectors
##                 1                66
```

Cross-validation chooses $\mathbf{C = 1}$, and the final model has **66 support vectors** (out of 427 training tumors). The CV accuracy is almost flat across the grid (from 0.927 to 0.937), so the exact choice of `C` matters little.

Note: the chosen C and the number of support vectors depend on the software, the grid, and the fold split. For example, `e1071::tune()` with `set.seed(42)` picks C = 100 with 62 support vectors; other values from correct code are fine. A larger cost gives fewer support vectors.

2. Generate predictions on the test set and report the confusion matrix and test accuracy.

```
cm_lin <- confusionMatrix(predict(svm_lin, test_s), test_s$diagnosis,
                          positive = "M")
cm_lin$table
```

```
##           Reference
## Prediction  B  M
##          B 86  6
##          M  3 47
```

```
round(cm_lin$overall["Accuracy"], 3)
```

```
## Accuracy
##    0.937
```

Test accuracy is **93.7%**: 6 malignant tumors are called benign and 3 benign tumors are called malignant.

A3. Non-linear (Radial) Kernel SVM (6 + 6 = 12 points)

1. Fit a support vector machine with a radial basis function kernel. Use 10-fold CV to jointly tune `cost` and `gamma` over grids of your choice. Report the chosen parameters.

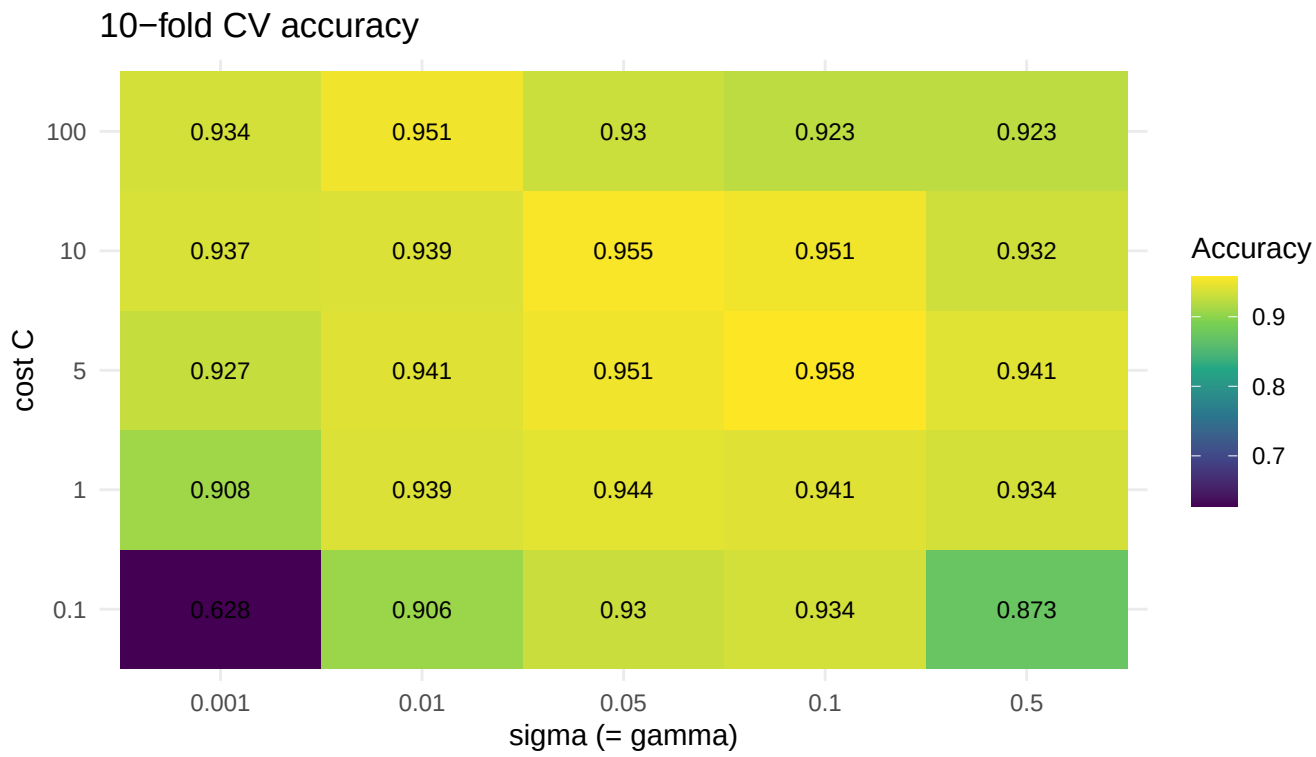
`caret`'s `svmRadial` calls `kernlab`, where the kernel is $K(x, x') = \exp(-\sigma\|x - x'\|^2)$. So `sigma` is the same

parameter as `gamma` in `e1071`.

```
set.seed(42)
svm_rad <- train(diagnosis ~ ., data = train_s, method = "svmRadial",
                 trControl = ctrl,
                 tuneGrid = expand.grid(C = c(0.1, 1, 5, 10, 100),
                                        sigma = c(0.001, 0.01, 0.05, 0.1, 0.5)))
svm_rad$bestTune
```

```
##    sigma C
## 14   0.1 5
```

```
ggplot(svm_rad$results, aes(factor(sigma), factor(C), fill = Accuracy)) +
  geom_tile() + geom_text(aes(label = round(Accuracy, 3)), size = 3) +
  scale_fill_viridis_c() +
  labs(x = "sigma (= gamma)", y = "cost C", title = "10-fold CV accuracy") +
  theme_minimal()
```



CV chooses **cost = 5** and **gamma (sigma) = 0.1**. The heat map shows a ridge of good settings: a larger cost goes with a smaller gamma. Very small gamma with small cost underfits, and the largest gamma overfits.

Note: the selected pair depends on the grid and the fold split. With `e1071::tune()` and the grid cost in {0.1, 1, 10, 100} × gamma in {0.001, 0.01, 0.1, 1}, the choice is cost = 100, gamma = 0.01.

2. Generate predictions on the test set, report the confusion matrix and test accuracy, and compare to the linear SVC from A2. Did the non-linear kernel improve performance? Given what you know about the geometry of this problem, is that what you'd expect?

```
cm_rad <- confusionMatrix(predict(svm_rad, test_s), test_s$diagnosis,
                          positive = "M")
cm_rad$table
```

```
##           Reference
## Prediction  B  M
##          B 89  6
##          M  0 47
```

```
tibble(model = c("Linear SVC", "Radial SVM"),
       test_accuracy = round(c(cm_lin$overall[["Accuracy"]],
                               cm_rad$overall[["Accuracy"]]), 3))
```

```
## # A tibble: 2 x 2
##   model      test_accuracy
##   <chr>              <dbl>
## 1 Linear SVC         0.937
## 2 Radial SVM         0.958
```

The radial SVM is slightly more accurate on the test set (95.8% vs. 93.7%). The difference is 3 of 142 tumors, and all of it comes from benign tumors. Both models miss the same number of cancers.

This is about what we would expect. Malignant nuclei are larger, more irregular, and more concave, and these features rise together, so the two classes are close to **linearly separable**. A straight boundary already does most of the work (Labs 5 and 6 showed the same thing). A flexible kernel can only add a small gain near the overlap, and a gain of 3 tumors on a test set of 142 is within what a different random split could change.

Note: results vary with the grid and the split, so "the radial kernel helped a little" and "it did not help" are both reasonable, as long as the answer refers to near-linear separability.

A4. Sensitivity and the Cost of Errors (4 + 3 = 7 points)

1. From a clinical perspective, a false negative (a malignant tumor classified as benign) is far more costly than a false positive. Report the sensitivity (recall for the malignant class) of both the linear and radial SVMs from A2/A3.

```
tibble(model       = c("Linear SVC", "Radial SVM"),
       sensitivity = c(cm_lin$byClass[["Sensitivity"]], cm_rad$byClass[["Sensitivity"]]),
       specificity = c(cm_lin$byClass[["Specificity"]], cm_rad$byClass[["Specificity"]]),
       missed_cancers = c(cm_lin$table["B", "M"], cm_rad$table["B", "M"])) |>
  mutate(across(c(sensitivity, specificity), \(x) round(x, 3)))
```

```
## # A tibble: 2 x 4
##   model      sensitivity specificity missed_cancers
##   <chr>            <dbl>       <dbl>          <int>
## 1 Linear SVC       0.887       0.966              6
## 2 Radial SVM       0.887       1                  6
```

Both models have sensitivity **0.887**: each misses 6 of the 53 malignant test tumors.

Note: set `positive = "M"` (or compute sensitivity by hand for the M class). caret's default positive class is the first level, B, so without it the reported "sensitivity" is for the benign class.

2. *Interpretation: Which model would you recommend for clinical deployment, and why — accuracy alone, or should sensitivity change your recommendation? Suggest one practical way to make an SVM more sensitive to the malignant class (e.g. adjusting the decision threshold or class weights).*

On this split the two models catch exactly the same cancers. The radial SVM's extra accuracy comes only from fewer false alarms. So **sensitivity does not separate them here**. The deciding questions are elsewhere:

- If the cost of a missed cancer dominates, as it does, neither model is good enough at the default boundary: missing about 1 in 9 cancers is too many for a screening tool.
- Between the two, I would recommend the **linear SVC** as the base model. It is as sensitive, simpler, more stable across data splits, and its weights show which nuclear features drive the call. The radial SVM is a defensible choice too, given its higher accuracy.

The more important step is to **move the operating point toward the malignant class**. Two practical ways, shown for the linear SVC:

```
# (1) Lower the probability threshold (Platt-scaled class probabilities)
set.seed(42)
svm_prob <- train(diagnosis ~ ., data = train_s, method = "svmLinear",
                  trControl = trainControl(method = "cv", number = 10, classProbs = TRUE),
                  tuneGrid = expand.grid(C = svm_lin$bestTune$C))
p_mal <- predict(svm_prob, test_s, type = "prob")[, "M"]
thr_tab <- map_dfr(c(0.5, 0.3, 0.2), function(t) {
  cm <- confusionMatrix(factor(ifelse(p_mal > t, "M", "B"), levels = c("B", "M")),
                        test_s$diagnosis, positive = "M")
  tibble(threshold = t, sensitivity = cm$byClass[["Sensitivity"]],
         specificity = cm$byClass[["Specificity"]])
})
# (2) Class weights: a violation on a malignant tumor costs 3 times more
svm_w <- svm(diagnosis ~ ., data = train_s, kernel = "linear",
             cost = svm_lin$bestTune$C, class.weights = c(B = 1, M = 3))
cm_w <- confusionMatrix(predict(svm_w, test_s), test_s$diagnosis, positive = "M")
bind_rows(thr_tab |> mutate(method = paste("threshold", threshold)),
          tibble(method = "class weights 1:3",
                 sensitivity = cm_w$byClass[["Sensitivity"]],
                 specificity = cm_w$byClass[["Specificity"]])) |>
  dplyr::select(method, sensitivity, specificity) |>
  mutate(across(-method, \(x) round(x, 3)))
```

```
## # A tibble: 4 x 3
##   method            sensitivity specificity
##   <chr>                   <dbl>       <dbl>
## 1 threshold 0.5           0.849       0.966
## 2 threshold 0.3           0.943       0.955
## 3 threshold 0.2           0.962       0.899
## 4 class weights 1:3       0.925       0.899
```

Lowering the threshold to 0.3, or weighting malignant errors 3:1, raises sensitivity above the default. It costs a little specificity: a few more benign tumors go on to biopsy, which is the trade a clinician would accept. The threshold or weight should be chosen on training data (by CV), not on the test set. We show test-set values here only to illustrate the trade-off.

Application, Part B: Clustering I — k-means and Cluster Evaluation (38 points)

Dataset: `Palmer Penguins`, saved as `penguins.csv`.

This dataset contains morphological measurements for `n = 344` penguins across three species (Adelie, Chinstrap, Gentoo) observed on islands in the Palmer Archipelago, Antarctica. We will ignore the species labels during clustering and use them only afterward, to evaluate how well unsupervised clustering

recovers the true species groupings.

- `species`: penguin species (Adelie, Chinstrap, Gentoo) — **label, do not use as a predictor**
- `island`: island of observation
- `bill_length_mm`, `bill_depth_mm`: bill dimensions in mm
- `flipper_length_mm`: flipper length in mm
- `body_mass_g`: body mass in grams
- `sex`: sex of the penguin

```
pacman::p_load(tidyverse, cluster, factoextra, mclust)
set.seed(42)
penguins <- read_csv("penguins.csv") |> drop_na()
features <- penguins |>
  dplyr::select(bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g)
features_scaled <- scale(features)
table(penguins$species)
```

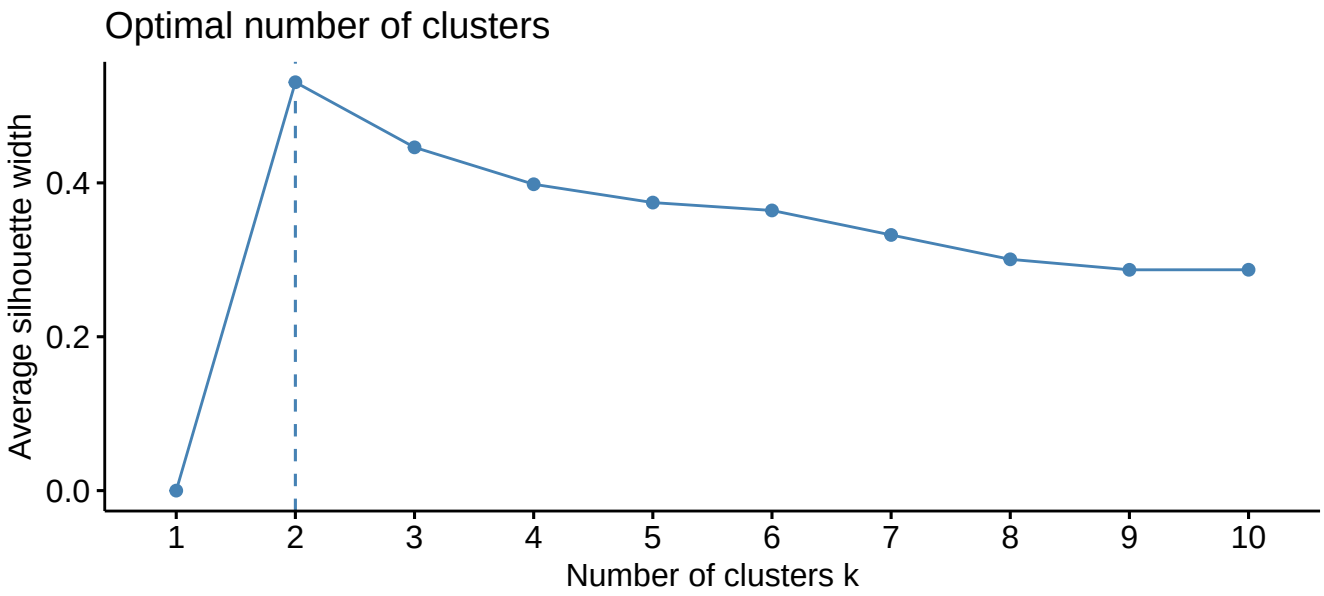
```
##
##    Adelie Chinstrap    Gentoo
##       146        68       119
```

`drop_na()` also drops the penguins with missing `sex`, so 333 of the 344 remain.

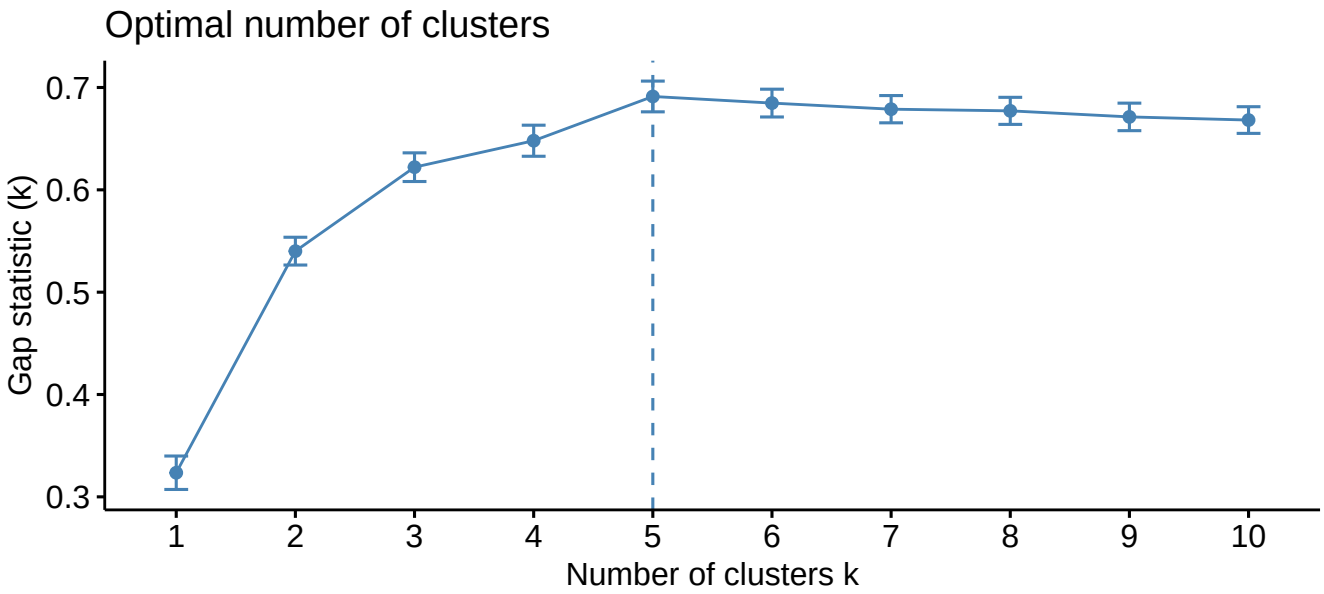
B1. Choosing k (6 + 6 = 12 points)

1. Using the four scaled morphological predictors, produce both a silhouette plot and a gap statistic plot (`factoextra::fviz_nbclust`) to help choose the number of clusters `k`. Report the value of `k` each method recommends.

```
set.seed(42)
p_sil <- fviz_nbclust(features_scaled, kmeans, method = "silhouette", nstart = 25)
set.seed(42)
p_gap <- fviz_nbclust(features_scaled, kmeans, method = "gap_stat",
                      nstart = 25, nboot = 50)
p_sil
```




```
p_gap
```




```
k_sil <- as.integer(as.character(p_sil$data$clusters[which.max(p_sil$data$y)]))
k_gap <- maxSE(p_gap$data$gap, p_gap$data$SE.sim, method = "firstSEmax")
c(silhouette = k_sil, gap = k_gap)
```

```
## silhouette        gap
##          2          5
```

The silhouette recommends $\mathbf{k = 2}$. The gap statistic (1-SE rule) recommends $\mathbf{k = 5}$.

Note: the silhouette choice of 2 is stable. The gap choice depends on `nboot` *and the seed; we get 5 for every seed we tried, but 4–6 from correct code is reasonable.*

2. There are three known species in this data. Do the silhouette and gap statistic recommendations agree with this, or with each other? Briefly comment on why an unsupervised method might not recover the "true" number of groups.

```
set.seed(42)
km2 <- kmeans(features_scaled, centers = 2, nstart = 25)
table(species = penguins$species, cluster_k2 = km2$cluster)
```

```
##            cluster_k2
## species       1   2
##   Adelie    146   0
##   Chinstrap  68   0
##   Gentoo      0 119
```

Neither matches the three species, and they do not agree with each other (2 vs. 5).

- **k = 2** splits Gentoo from everyone else. Adelie and Chinstrap have very similar body size, flippers, and bill depth; they differ mainly in bill length. With four variables on equal footing, the Gentoo split is by far the largest gap in the data, so the silhouette (which rewards compact, well-separated clusters) prefers it.
- **k = 5** finds structure *inside* species. Penguins are sexually dimorphic: males are heavier with longer bills, so each species is itself two overlapping sub-clouds. The gap statistic sees real structure beyond 3 groups. That structure is sex, not species.

The general point: clustering finds whatever groups dominate the geometry of the chosen variables. "Species" is one valid grouping, and others exist (sex, or species × sex). The algorithm cannot know which grouping we care about. The "true" number of clusters is defined by the scientific question, not by the data alone.

B2. K-Means Clustering (5 + 6 = 11 points)

1. Perform k-means clustering with `centers = 3` on the scaled features (use `nstart = 25`). Create a cross-tabulation (`table()`) comparing the true `species` labels to the cluster assignments.

```
set.seed(42)
km3 <- kmeans(features_scaled, centers = 3, nstart = 25)
tab_km <- table(species = penguins$species, cluster = km3$cluster)
tab_km
```

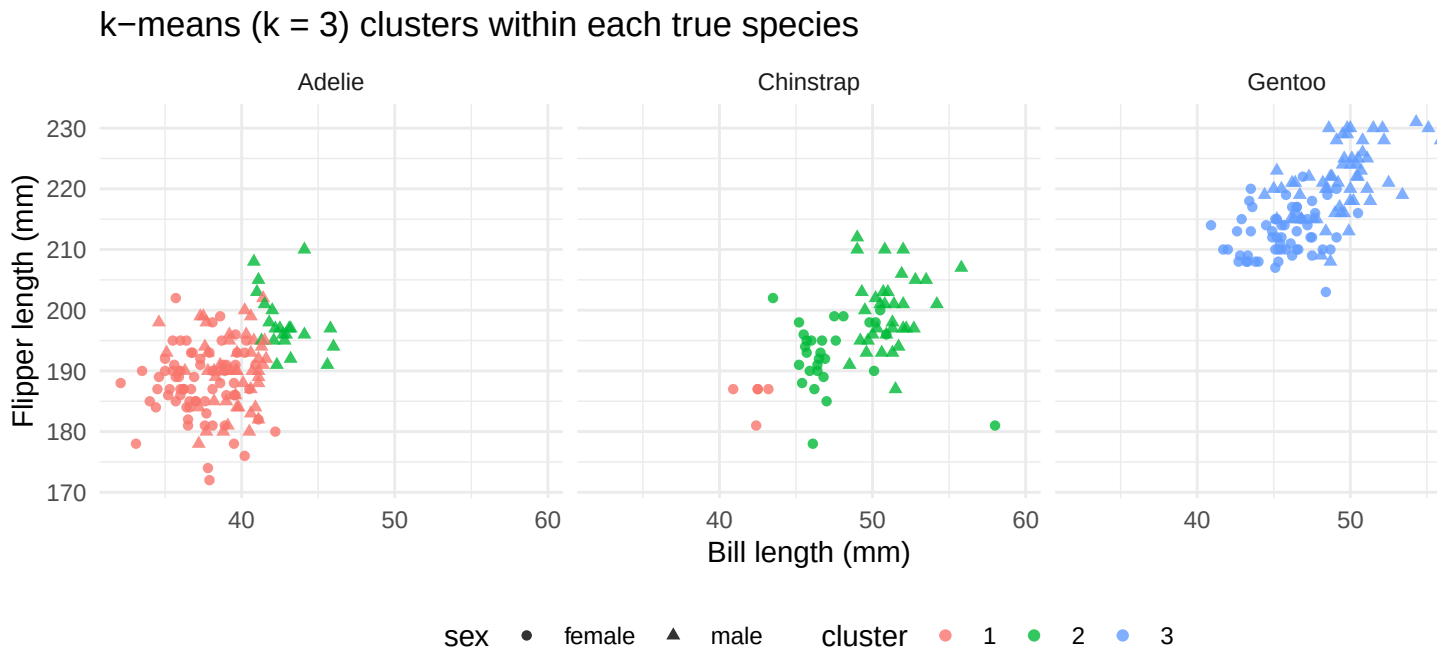
```
##            cluster
## species       1   2   3
##   Adelie    124  22   0
##   Chinstrap   5  63   0
##   Gentoo      0   0 119
```

(Cluster numbers are arbitrary labels. A different seed can permute them.)

2. Make a scatterplot of `bill_length_mm` vs. `flipper_length_mm`, colored by cluster assignment and faceted (or shaped) by true species. Does k-means recover the species groups well? Which species, if any, gets confused with another?

```
penguins |>
  mutate(cluster = factor(km3$cluster)) |>
  ggplot(aes(bill_length_mm, flipper_length_mm, colour = cluster, shape = sex)) +
  geom_point(alpha = 0.8) +
```

```
  facet_wrap(~ species) +
  labs(x = "Bill length (mm)", y = "Flipper length (mm)",
       title = "k-means (k = 3) clusters within each true species") +
  theme_minimal() + theme(legend.position = "bottom")
```




```
table(species = penguins$species, cluster = km3$cluster, sex = penguins$sex)
```

```
## , , sex = female
##
##            cluster
## species      1  2  3
##   Adelie    73  0  0
##   Chinstrap  5 29  0
##   Gentoo     0  0 58
##
## , , sex = male
##
##            cluster
## species      1  2  3
##   Adelie    51 22  0
##   Chinstrap  0 34  0
##   Gentoo     0  0 61
```

k-means recovers the species **partly**:

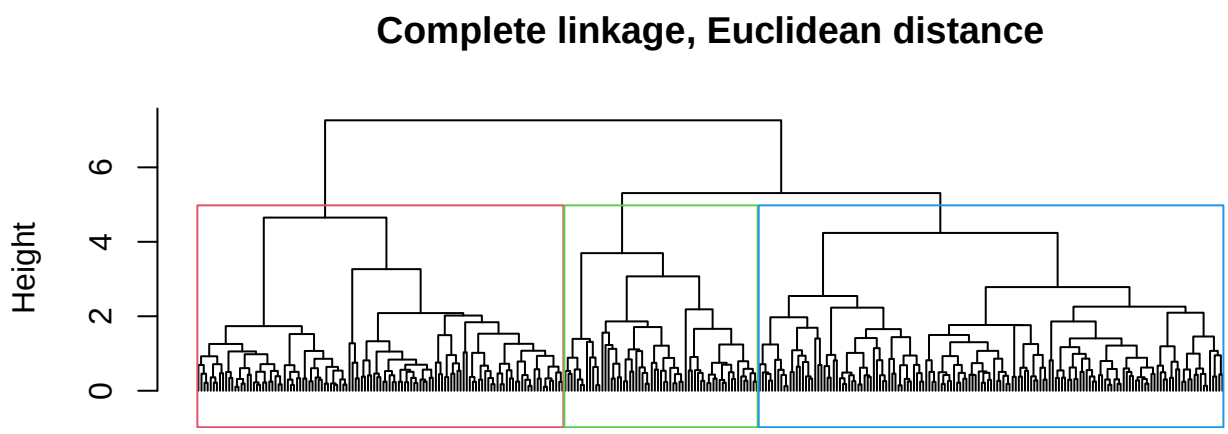
- **Gentoo** is recovered perfectly (all 119 in their own cluster).
- **Adelie and Chinstrap are confused.** 22 Adelie fall in the mostly-Chinstrap cluster, and 5 Chinstrap fall in the Adelie cluster. The 3-way table shows a pattern: all 22 misplaced Adelie are **males**, and all 5 misplaced Chinstrap are **females**. Large male Adelie and small female Chinstrap overlap in size, and k-means groups by overall size.

Overall, 306 of 333 penguins (92%) are in the cluster that matches their species (adjusted Rand index 0.8).

B3. Hierarchical Clustering & Comparison (7 + 8 = 15 points)

1. Perform hierarchical clustering on the scaled features using complete linkage on a Euclidean distance matrix, and cut the dendrogram into `k = 3` clusters. Create a cross-tabulation comparing true species to these cluster assignments.

```
hc <- hclust(dist(features_scaled, method = "euclidean"), method = "complete")
hc3 <- cutree(hc, k = 3)
plot(hc, labels = FALSE, hang = -1, main = "Complete linkage, Euclidean distance",
     xlab = "", sub = "")
rect.hclust(hc, k = 3, border = 2:4)
```




```
tab_hc <- table(species = penguins$species, cluster = hc3)
tab_hc
```

```
##            cluster
## species       1   2   3
##   Adelie    145   1   0
##   Chinstrap   6  62   0
##   Gentoo      0   0 119
```

2. *Interpretation: Compare the k-means and hierarchical clustering solutions using their cross-tabulations against the true species labels — which method recovered the species structure more accurately? Given that k-means assumes roughly spherical, equal-variance clusters, does that assumption seem reasonable for this data? What does this exercise suggest about the general risk of trusting unsupervised cluster labels as if they were ground truth?*

```
tibble(method = c("k-means", "hierarchical (complete)"),
       matched = c(sum(apply(tab_km, 2, max)), sum(apply(tab_hc, 2, max))),
       ARI = c(adjustedRandIndex(km3$cluster, penguins$species),
               adjustedRandIndex(hc3, penguins$species))) |>
```

```
  mutate(percent = round(100 * matched / nrow(penguins), 1), ARI = round(ARI, 3))
```

```
## # A tibble: 2 x 4
##   method                   matched   ARI percent
##   <chr>                      <int> <dbl>   <dbl>
## 1 k-means                      306 0.799    91.9
## 2 hierarchical (complete)      326 0.943    97.9
```

Hierarchical clustering does better here. It misplaces only 7 penguins (1 Adelie and 6 Chinstrap), against 27 for k-means (ARI 0.94 vs. 0.8).

The k-means assumption is doubtful. k-means draws straight boundaries halfway between centroids, which works best for round clusters of similar spread and size. Here:

- the species groups have different sizes (146, 68, 119) and different shapes after scaling;
- each species is two sub-clouds (females and males), so a species cluster is elongated along a "body size" direction instead of being round;
- Adelie and Chinstrap overlap. A halfway boundary then cuts through the large male Adelie.

Complete linkage does not assume round clusters, and here it follows the species better. This is specific to this data set: complete linkage is not better in general (in Lab 2 it did worse than k-means on the thyroid data).

The general risk. Both methods return a confident label for every penguin, with no warning when the label is wrong. Two reasonable methods disagree on about 22 penguins, and the silhouette and gap statistic disagree on how many groups exist. Cluster labels depend on the variables, the scaling, the algorithm, the linkage, and k. They are a hypothesis to be checked against domain knowledge, not ground truth. Here we could check them only because species labels were available. In a real unsupervised problem, we could not.

Part C: Why not cross-validate K? A simulation study (15 points)

In Part A, and in earlier assignments, you chose tuning parameters (the SVM cost, γ) by cross-validation. For k-means, the lecture and lab used the elbow, the silhouette, and the gap statistic instead. In this part you find out why, with a simulation where you know the true answer.

A natural way to cross-validate k-means is this. Split the data into folds. For each fold, fit k-means on the other folds. Then compute the **held-out WSS**: the sum, over the held-out points, of the squared Euclidean distance from each point to its **nearest training centroid**. Average over the folds, repeat for each K, and choose the K with the smallest average held-out WSS.

This part is **open-ended**. There is no single correct design. You are graded on a clear, reasoned design, correct code, and a careful explanation, not on matching a particular number. Keep the study small: it should run in under a minute.

The helper below computes the held-out WSS for one fold. You may use it or write your own.

```
# Held-out WSS: sum of squared distances from each test point to its nearest
# training centroid. `centers` is the K x p matrix from kmeans()$centers.
heldout_wss <- function(test, centers) {
  test <- as.matrix(test)
  d2 <- sapply(seq_len(nrow(centers)),
               function(k) colSums((t(test) - centers[k, ])^2))
  sum(apply(matrix(d2, nrow = nrow(test)), 1, min))
}
```

*Note for Part C: what follows is **one** valid answer. Other designs, other fixes, and other "changes" are equally valid; the reasoning matters more than the numbers.*

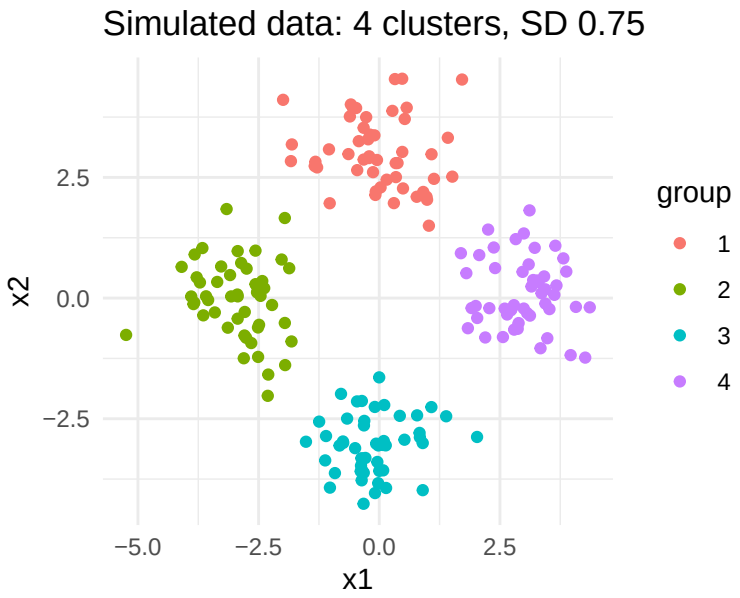
C1. Run the cross-validation (5 points)

Simulate a data set with a known number of clusters K_{true}. You choose n, the number of variables, K_{true}, and how well separated the clusters are. State your choices and say why you made them. Then run 5-fold cross-validation of the held-out WSS for $K = 1, \ldots, 10$ (use `nstart = 10` or more). Plot the average held-out WSS against K. Which K does the "smallest held-out WSS" rule choose?

Design. $K_{\text{true}} = 4$ round Gaussian clusters in $p = 2$ dimensions, 50 points each ($n = 200$). The centres sit on a circle of radius 3, and each cluster has SD 0.75. Reasons:

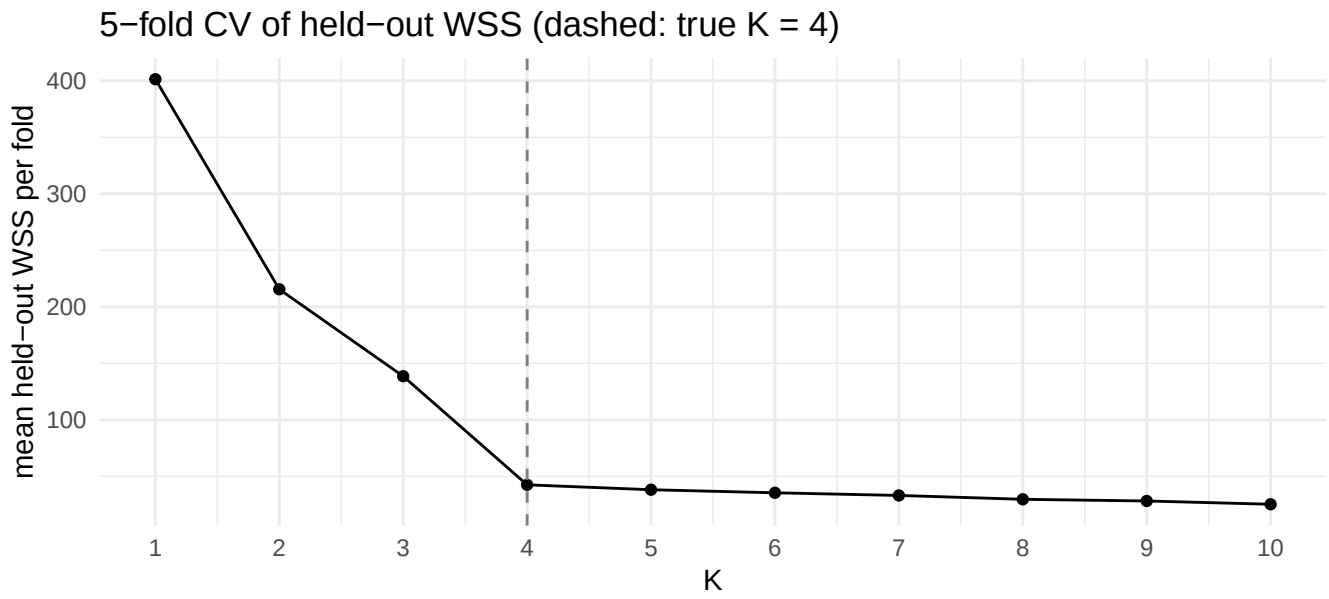
- Round, equal-size, equal-spread clusters are exactly what k-means assumes. If the CV rule fails here, it fails because of the rule, not because of a hard data set.
- Two dimensions let us plot the data and see that 4 is the obvious answer.
- $K_{\text{true}} = 4$ (not 2) leaves room to see both under- and over-estimation.
- SD 0.75 gives clear separation. In C3 we raise it to make the clusters overlap.

```
sim_clusters <- function(K = 4, n_per = 50, sd = 0.75, seed = 42) {
  set.seed(seed)
  ang <- 2 * pi * (1:K) / K
  g   <- rep(1:K, each = n_per)
  data.frame(x1 = rnorm(K * n_per, 3 * cos(ang)[g], sd),
             x2 = rnorm(K * n_per, 3 * sin(ang)[g], sd),
             group = factor(g))
}
sim <- sim_clusters()
ggplot(sim, aes(x1, x2, colour = group)) + geom_point() + coord_equal() +
  labs(title = "Simulated data: 4 clusters, SD 0.75") + theme_minimal()
```




```
cv_heldout <- function(X, Ks = 1:10, n_folds = 5, nstart = 10, seed = 42) {
  X <- as.matrix(X)
  set.seed(seed)
  fold <- sample(rep(1:n_folds, length.out = nrow(X)))
  sapply(Ks, function(k) {
    mean(sapply(1:n_folds, function(f) {
```

```
      fit <- kmeans(X[fold != f, ], centers = k, nstart = nstart)
      heldout_wss(X[fold == f, , drop = FALSE], fit$centers)
    }))
  })
}
cv_wss <- cv_heldout(sim[, c("x1", "x2")])
cv_tab <- tibble(K = 1:10, heldout_wss = cv_wss)
ggplot(cv_tab, aes(K, heldout_wss)) + geom_line() + geom_point() +
  geom_vline(xintercept = 4, linetype = 2, colour = "grey50") +
  scale_x_continuous(breaks = 1:10) +
  labs(y = "mean held-out WSS per fold",
       title = "5-fold CV of held-out WSS (dashed: true K = 4)") +
  theme_minimal()
```




```
round(cv_wss, 1)
```

```
##  [1] 401.4 215.5 138.6  42.5  38.1  35.4  33.1  29.7  28.2  25.3
```

The held-out WSS **falls at every** K, so the "smallest held-out WSS" rule chooses $\mathbf{K = 10}$, the largest value we tried. It would choose an even larger K if we let it.

C2. Explain the result (4 points)

Why does held-out WSS behave this way as K grows? Compare with cross-validating the SVM cost in Part A: what makes the cross-validated error there go back up for very flexible models, and what is missing in the clustering problem? Does your plot still contain useful information about K_{true}?

- **Why it keeps falling.** Held-out WSS measures how close new points are to the nearest centroid. More centroids cover the space more densely, so a new point is almost always closer to *some* centroid, even if the extra centroids only split a real cluster in two. Splitting a cluster never makes the nearest-centroid distance worse for new data from that cluster. So the loss keeps falling with K, on held-out data as well as on training data. k-means with many centroids is a good way to *compress* data, and compression always improves with more centroids.
- **Why SVM CV works.** In Part A the held-out loss is a prediction error for a **label** y. A very flexible

SVM (large cost, large gamma) bends its boundary around the noise in the training labels. Those twists do not carry over to new tumors, so held-out error goes back up. That is the U-shape (bias-variance trade-off) that CV finds.

- **What is missing in clustering.** There is no label, so there is no noise to overfit, and no penalty for using too many clusters. The held-out WSS rewards flexibility and never punishes it.
- **Useful information.** The plot still has an **elbow**: held-out WSS drops steeply up to $K = 4$ (from 401 to 42) and falls slowly after that (only to 25 at $K = 10$). The held-out curve is just the elbow plot again, with the same strengths and the same weakness (the bend is a judgement).

C3. Fix it (4 points)

Propose and implement a procedure that uses held-out data or resampling and **does** recover K_{true} in your simulation. Some ideas, which you may use or ignore:

- **Stability:** cluster two overlapping subsamples, and measure how well the two clusterings agree on the shared points (e.g. with the adjusted Rand index, `mclust::adjustedRandIndex()`). Real clusters come back each time; splits made up by the algorithm do not.
- **Prediction strength** (Tibshirani and Walther, 2005): cluster a training half and a test half separately. Check how often pairs of test points that are in the same test cluster are also assigned to the same training centroid.
- **A reference comparison:** compare held-out WSS with its value on data that have no clusters, as the gap statistic does.

Show that your procedure finds K_{true}. Then change one thing in your simulation (for example, make the clusters overlap more, or make them unequal in size) and report what happens to your procedure.

Procedure: resampling stability. For each K, repeat 20 times: draw two random 80% subsamples, run k-means on each, and compute the adjusted Rand index (ARI) between the two clusterings on the points both subsamples share. Choose the K with the highest mean ARI.

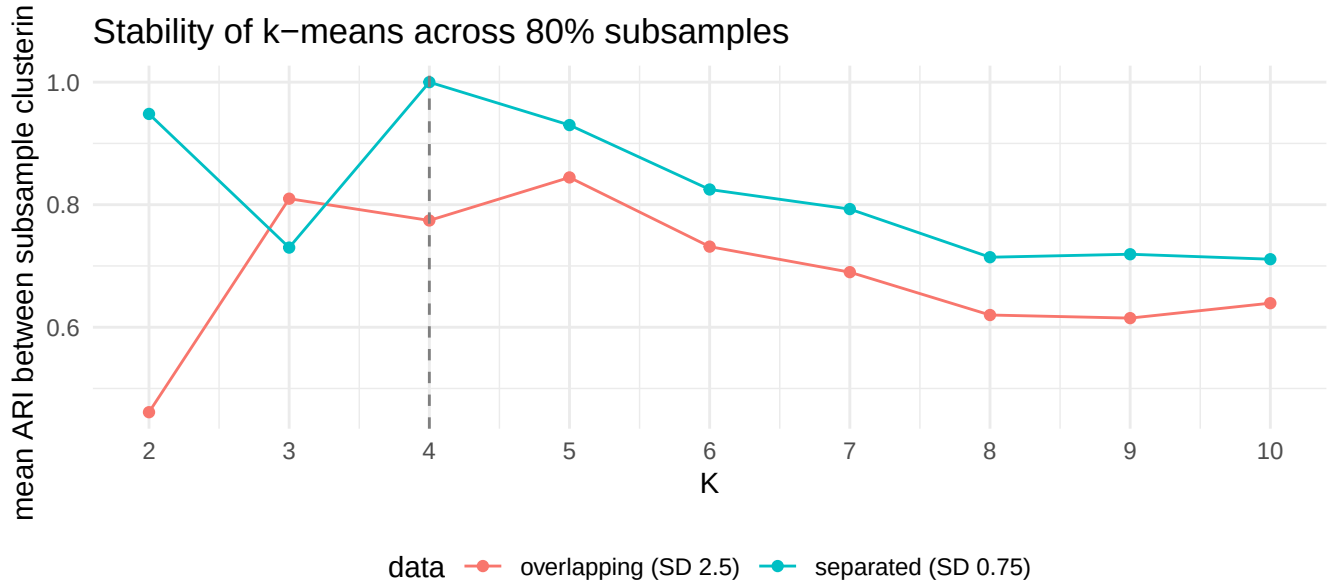
The idea: when K matches real structure, every subsample finds the same clusters, so ARI is close to 1. When K is too large, k-means must split a real cluster, and *where* it splits changes from subsample to subsample, so the two clusterings disagree. Too small a K forces a merge of real clusters, and which ones merge can also change. Unlike held-out WSS, stability can get worse as K grows.

```
stability <- function(X, Ks = 2:10, reps = 20, frac = 0.8, nstart = 10, seed = 42) {
  X <- as.matrix(X); n <- nrow(X)
  set.seed(seed)
  sapply(Ks, function(k) {
    mean(replicate(reps, {
      a <- sample(n, frac * n); b <- sample(n, frac * n)
      shared <- intersect(a, b)
      cl_a <- kmeans(X[a, ], k, nstart = nstart)$cluster
      cl_b <- kmeans(X[b, ], k, nstart = nstart)$cluster
      adjustedRandIndex(cl_a[match(shared, a)], cl_b[match(shared, b)])
    }))
  })
}
stab_sep <- stability(sim[, c("x1", "x2")])
```

Change: make the clusters overlap. Same design, but the SD goes from 0.75 to 2.5. The cluster centres are only about 4 units apart, so at this SD the four groups merge into one round cloud.

```
sim_ovl  <- sim_clusters(sd = 2.5)
stab_ovl <- stability(sim_ovl[, c("x1", "x2")])
cv_ovl   <- cv_heldout(sim_ovl[, c("x1", "x2")])
```

```
stab_tab <- bind_rows(tibble(K = 2:10, ARI = stab_sep, data = "separated (SD 0.75)"),
                      tibble(K = 2:10, ARI = stab_ovl, data = "overlapping (SD 2.5)"))
ggplot(stab_tab, aes(K, ARI, colour = data)) + geom_line() + geom_point() +
  geom_vline(xintercept = 4, linetype = 2, colour = "grey50") +
  scale_x_continuous(breaks = 2:10) +
  labs(y = "mean ARI between subsample clusterings",
       title = "Stability of k-means across 80% subsamples") +
  theme_minimal() + theme(legend.position = "bottom")
```




```
c(separated = (2:10)[which.max(stab_sep)], overlapping = (2:10)[which.max(stab_ovl)],
  heldout_WSS_overlapping = which.min(cv_ovl))
```

```
##                separated             overlapping heldout_WSS_overlapping
##                        4                       5                      10
```

- **Separated clusters:** stability peaks at $\mathbf{K = 4}$ (mean ARI 1.00). It recovers K_{true}, where held-out WSS chose 10.
- **Overlapping clusters:** stability now picks $\mathbf{K = 5}$, but the curve has no clear peak: $K = 3$, 4, and 5 all score between 0.77 and 0.84. The data no longer contain four separate groups, and the procedure correctly finds no K that stands out. A different seed could pick any of 3, 4, or 5. Held-out WSS still picks 10, so it is wrong in both settings.

So the fix works when the clusters are clear. It keeps working for a fair amount of overlap (we also tried SD 1.5, 1.75, and 2, and it still found $K = 4$), and fails only when the groups have really merged. Even then it fails "honestly": the flat curve warns us that no K is well supported. One limitation remains. Like the silhouette, stability starts at $K = 2$, so it can never say "there are no clusters". The gap statistic can.

Note: a fix that fails in the "changed" setting is still a good answer, as long as the failure is reported and explained.

C4. Recommendation (2 points)

In 3–4 sentences, written for a collaborator with no statistics training: why can't we choose the number of clusters the same way we choose a tuning parameter for a classifier, and what would you do instead?

When we tune a classifier, we can check its predictions against known answers in data it has not seen, and an over-complicated model gets caught because it predicts worse. In clustering there are no known answers to check against, and the obvious score, how close each point is to a group centre, always improves when we add more groups, so it would always tell us to use more groups. Instead, I would look at several measures that ask whether the groups are real (whether they reappear when we re-run the analysis on different subsets of patients, and how well separated they are), and then choose among the few candidates by asking which grouping makes sense for your question. The final choice of the number of groups should be a joint decision between us, not a number the software picks alone.